

Windows Custom PNP Drivers & VirtIO Injection

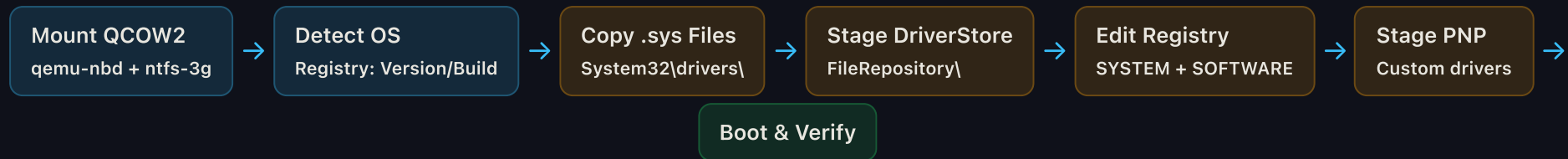
Deterministic Offline Driver Deployment for VMware-to-KVM Migration

The complete Windows driver pipeline: offline VirtIO injection with registry editing, custom PNP driver staging via pnputil, network configuration preservation, multi-stage boot verification, and VMware Tools removal. All applied offline via qemu-nbd — zero BSOD, zero manual intervention.

6 VirtIO Drivers | Custom PNP | Network Preserve | Registry Edit | Multi-Stage Boot | Proprietary (HyperSDK)

Driver Injection Architecture

How hyper2kvm deterministically prepares Windows for KVM — entirely offline.



Stage 1: Offline Pre-Injection

- Mount disk via qemu-nbd + ntfs-3g
- Detect OS from registry (ProductName, CurrentBuild)
- Map architecture (x86 vs amd64)
- Copy VirtIO .sys files to System32\drivers\
- Stage INF+SYS in DriverStore\FileRepository\
- Register OEM INF files in Windows\INF\
- Edit SYSTEM hive (services, CriticalDeviceDatabase)
- Edit SOFTWARE hive (RunOnce, OOBE bypass)
- Disable VMware services in registry
- Stage custom PNP drivers to C:\hyper2kvm-drivers

Stage 2: First Boot

- Windows boots with VirtIO storage (Start=0)
- Auto-login enabled (no password prompt)
- Win11 OOBE bypass (BypassNRO, SkipMachineOOBE)
- RunOnce batch file executes:
 1. pnputil installs VirtIO from ISO
 2. pnputil installs custom PNP drivers
- Network apply script runs (PowerShell)
- QEMU guest agent MSI installed (optional)
- Serial console available on COM1

Stage 3: Verification

- Automated boot test (no BSOD check)
- Guest agent ping confirms drivers installed
- Offline verification via NBD remount
- Check: viostor.sys present & loaded
- Check: netkvm.sys present
- Check: balloon.sys present
- Network connectivity verified
- RDP accessible on port 3389
- Generate production domain XML

Key innovation: All critical drivers are injected offline via direct registry editing. The VM boots with VirtIO storage already functional (Start=0). pnputil on first boot handles the PNP binding — but the boot itself never fails because the boot-critical driver is already registered.

VirtIO Drivers **Injected**

Six VirtIO drivers automatically injected with full registry configuration.

Driver	File	Purpose	Start	Group	PCI Hardware IDs
viostor	viostor.sys	VirtIO block storage (boot-critical)	0 (Boot)	SCSI miniport	PCI\VEN_1AF4&DEV_1001, PCI\VEN_1AF4&DEV_1042
vioscsi	vioscsi.sys	VirtIO SCSI controller	0 (Boot)	SCSI miniport	PCI\VEN_1AF4&DEV_1004, PCI\VEN_1AF4&DEV_1048
NetKVM	netkvm.sys	VirtIO network adapter	2 (Auto)	NDIS	PCI\VEN_1AF4&DEV_1000, PCI\VEN_1AF4&DEV_1041
Balloon	balloon.sys	Dynamic memory management	3 (Demand)	Extended Base	PCI\VEN_1AF4&DEV_1002, PCI\VEN_1AF4&DEV_1045
vioserial	vioser.sys	Serial port / guest agent channel	3 (Demand)	Extended Base	PCI\VEN_1AF4&DEV_1003, PCI\VEN_1AF4&DEV_1043
viorng	viorng.sys	Hardware RNG from KVM host	3 (Demand)	Extended Base	PCI\VEN_1AF4&DEV_1005, PCI\VEN_1AF4&DEV_1044

Registry Operations (SYSTEM Hive)

Services Registration

```
HKLM\SYSTEM\ControlSet001\Services\viostor
Type      = DWORD:1    (kernel driver)
Start     = DWORD:0    (boot-start)
ErrorControl = DWORD:1  (normal)
ImagePath = "system32\drivers\viostor.sys"
Group     = "SCSI miniport"

HKLM\SYSTEM\ControlSet001\Services\netkvm
Type      = DWORD:1
Start     = DWORD:2    (auto-start)
ImagePath = "system32\drivers\netkvm.sys"
Group     = "NDIS"
```

CriticalDeviceDatabase

```
HKLM\SYSTEM\ControlSet001\Control\
CriticalDeviceDatabase\
  PCI#VEN_1AF4&DEV_1001
    ClassGUID = "{4D36E97B-...}"
    Service   = "viostor"
  PCI#VEN_1AF4&DEV_1042
    ClassGUID = "{4D36E97B-...}"
    Service   = "viostor"
  PCI#VEN_1AF4&DEV_1000
    ClassGUID = "{4D36E972-...}"
    Service   = "netkvm"
```

VMware Services Disabled in Registry

Service	Description	Action
VMTools	VMware Tools service	Start = DWORD:4 (Disabled)
vm3dservice	VMware 3D graphics	Start = DWORD:4 (Disabled)
VGAAuthService	VMware Guest Authentication	Start = DWORD:4 (Disabled)

vmvss	VMware VSS provider	Start = DWORD:4 (Disabled)
vmci	VMware VMCI driver	Start = DWORD:4 (Disabled)
vsock	VMware vSock driver	Start = DWORD:4 (Disabled)
vmhgfs	VMware Shared Folders	Start = DWORD:4 (Disabled)
vmrawdsk	VMware Raw Disk driver	Start = DWORD:4 (Disabled)
vmusbmouse	VMware USB mouse	Start = DWORD:4 (Disabled)

Custom PNP Driver Injection

Stage any vendor-supplied driver for automatic PNP installation on first boot.

How It Works

- Accepts directories or ISOs as driver sources
- ISOs are auto-mounted and contents copied
- All driver files staged to `C:\hyper2kvm-drivers\`
- RunOnce registry key triggers pnputil on first boot
- pnputil recursively discovers and installs all INF files
- Supports multiple driver packages (repeatable flag)
- Works alongside VirtIO driver injection

Use Cases

- **GPU passthrough:** NVIDIA/AMD vendor drivers
- **Storage HBA:** Fibre Channel, iSCSI drivers
- **Network:** Mellanox, Intel vendor NICs
- **Smart cards:** Security hardware drivers
- **Printers:** Enterprise printer drivers
- **USB devices:** Specialized peripherals
- **Custom hardware:** Any INF-based driver

Configuration

YAML Config

```
# migration.yaml
custom_pnp_drivers:
  - /opt/drivers/nvidia-gpu/
  - /opt/drivers/mellanox-nic.iso
  - /opt/drivers/vendor-storage/
```

CLI

```
h2kvmctl --config migration.yaml \
  --custom-pnp-drivers /opt/nvidia-gpu/ \
  --custom-pnp-drivers /opt/mellanox.iso \
  --custom-pnp-drivers /opt/vendor-hba/
```

RunOnce Batch File (Staged in Registry)

```
@echo off
rem === VirtIO driver installation from ISO ===
for %%d in (D E F G H I J K L M N) do (
  if exist "%%d:\viosstor" ( set ISO=%%d: & goto :found )
)
:found
rem Install: viostor NetKVM Balloon vioserial viorng qxldod viogpudo vioinput viofs pvpanic
for %%c in (viostor NetKVM Balloon vioserial viorng qxldod viogpudo vioinput viofs pvpanic smbush qemufwcfg) do (
  if exist "%ISO%\%%c\%OSFOLDER%\%ARCH%" (
    pnputil /add-driver "%ISO%\%%c\%OSFOLDER%\%ARCH%\*.inf" /install >nul 2>&1
  )
)
rem Install QEMU guest agent
if exist "%ISO%\guest-agent\qemu-ga-x86_64.msi" (
  msixexec /i "%ISO%\guest-agent\qemu-ga-x86_64.msi" /quiet /norestart
)
```

```
rem === Custom PNP driver installation ===  
if exist "C:\hyper2kvm-drivers" (  
    pnputil /add-driver "C:\hyper2kvm-drivers\*.inf" /subdirs /install  
)
```

pnputil /subdirs recursively scans all subdirectories for INF files. This means you can drop an entire vendor driver package (with multiple architectures and versions) and pnputil will find and install the correct one for the running OS.

Windows Network Preservation

Snapshot existing TCP/IP config from VMware, apply to VirtIO NIC on KVM — zero manual IP reconfiguration.

Offline Registry Read
Tcpip\Parameters\Interfaces



Snapshot to JSON
saved_network.json



Stage Override
network_override.json



First Boot Apply
PowerShell script

What Gets Captured

- IP addresses (static and DHCP-assigned)
- Subnet masks
- Default gateways
- DNS servers (static and DHCP-derived)
- DNS suffix / domain
- Interface profiles
- DHCP vs Static mode per interface
- All data from offline SYSTEM registry hive

Three Configuration Methods

- **Auto-preserve:** Snapshot VMware config, apply to KVM NIC
- **Override file:** `--win-net-override /path/to/override.json`
- **Inline JSON:** `--win-net-json '{"schema":1,...}'`
- **YAML native:** `win_net:` section in config
- Adapter selection by MAC address or name
- Supports both DHCP and static modes
- Optional DNS override in DHCP mode

Configuration Examples

Static IP (YAML)

```
win_net:
  schema: 1
  mode: static
  static:
    ip: 192.168.1.50
    mask: 255.255.255.0
    gateway: 192.168.1.1
  dns_servers:
    - "1.1.1.1"
    - "8.8.8.8"
```

DHCP + Custom DNS

```
win_net:
  schema: 1
  mode: dhcp
  dhcp:
    dns_servers:
      - "10.0.0.53"
      - "10.0.0.54"
```

MAC-Based Selection

```
win_net:
  schema: 1
  mode: static
  adapter:
    mac: "52-54-00-12-34-56"
  static:
    ip: 10.73.1.50
    mask: 255.255.255.0
    gateway: 10.73.1.1
  dns_servers:
    - "10.73.1.53"
```

Files Staged in Guest

Path	Purpose
C:\hyper2kvm\net\saved_network.json	Snapshot of VMware TCP/IP config from offline registry

C:\hyper2kvm\net\network_override.json	User-provided override config (optional)
C:\hyper2kvm\net\apply-network.ps1	PowerShell script that applies config on first boot
C:\hyper2kvm\net\apply-network.log	Execution log for troubleshooting

Multi-Stage VirtIO Boot

The `--virtio-deploy-boot` flag enables a 4-stage automated deployment for maximum reliability.

Stage 1: Bootstrap Boot

- SATA disk (safe, always works)
- VirtIO NIC (triggers netkvm driver)
- VirtIO balloon + serial + guest agent
- VirtIO ISO attached as CD-ROM
- UEFI firmware + NVRAM
- OOB auto-bypass configured
- RunOnce batch installs all VirtIO drivers via pnputil
- Custom PNP drivers installed via pnputil

Stage 2: Wait for Guest Agent

- Polls `virsh qemu-agent-command "guest-ping"`
- Default timeout: 180 seconds (configurable)
- Guest agent confirms RunOnce has executed
- All drivers installed and bound to devices
- Network operational (DHCP or static applied)
- VM is functional and accessible

Stage 3: Offline Verification

- Shutdown VM cleanly
- Remount QCOW2 via NBD
- Verify critical files present:
 - ✓ `viostor.sys` (critical)
 - ✓ `netkvm.sys` (optional)
 - ✓ `balloon.sys` (optional)
 - ✓ `vioser.sys` (optional)
- Fail fast if boot-critical driver missing

Stage 4: Production VM

- Generate production libvirt domain XML
- VirtIO disk + VirtIO NIC
- Q35 machine type with PCIe
- UEFI/BIOS as detected
- Optional auto-start final VM
- KubeVirt manifest (if `deploy_k8s: true`)
- Webhook notification on completion

Configuration

```
# Enable multi-stage VirtIO boot deployment
virtio_deploy_boot: true
virtio_deploy_timeout: 300          # Wait up to 5 minutes for guest agent
no_virtio_deploy_start_final: false # Auto-start production VM

# VirtIO ISO source
virtio_win_iso: /usr/share/virtio-win/virtio-win.iso

# Optional custom drivers (staged alongside VirtIO)
custom_pnp_drivers:
  - /opt/vendor-gpu-drivers/
```

```
- /opt/vendor-storage.iso
```

```
# Network preservation
win_net:
  schema: 1
  mode: static
  static:
    ip: 10.0.1.50
    mask: 255.255.255.0
    gateway: 10.0.1.1
    dns_servers: ["10.0.1.53"]
```

Zero-Touch Windows Migration

Offline VirtIO injection + custom PNP staging + network preservation + automated boot verification.

No manual driver installation. No BSOD. No IP reconfiguration. No VMware remnants.